

LAB ACTION PLAN FOR WEEK 2

Objective:

1. Pushing multi-folder project into public repository (by student).
2. Students must explore all listed git commands on the multi-folder project in local and remote repository.
3. Students must explore all git commands on given scenario-based question.
4. Students need to make note in observation book and also upload the executed exercise on scenario-based question.

Task 1: Pushing multi-folder project into private repository(Note: student take screenshots for upload).

1. Create a Public Repository on GitHub

- Go to <https://github.com>
- Click **New repository**
- Give it a name
- Check **Public**
- Click **Create repository**

2. Initialize Git in Your Local Project (if not already done)

Open a terminal in the **root** of your multi-folder project:

```
cd /path/to/your/project  
git init
```

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2  
$ git init  
Initialized empty Git repository in C:/SE Lab Internal Practice Week 2/.git/
```

3. Add the Remote Repository (in bash)

You'll see a URL on GitHub after creating the repo. It will look as below, run this command in git bash:

```
git remote add origin https://github.com/yourusername/your-public-repo.git
```

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)  
$ git remote add origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice.git
```

4. Add and Commit Your Files(in bash)

```
git add .
```

```
git commit -m "Initial commit"
```

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)
$ nano file1.txt

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)
$ nano file2.txt

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)
$ git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
    add

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)
$ git add .
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'file2.txt', LF will be replaced by CRLF the next time Git touches it

```

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)
$ git branch -M main

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git commit -m "First commit"
[main (root-commit) ab957a8] First commit
2 files changed, 4 insertions(+)
create mode 100644 file1.txt
create mode 100644 file2.txt

```

5. Push to the Private Repository

If the repo is new and empty, push like this from git bash:

git push -u origin master

If you're using the main branch (GitHub default):

git push -u origin main

You may be prompted to log in (via username/password or token).

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 32 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 296 bytes | 296.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

Task 2: Students must explore all listed git commands on the multi-folder project in local and remote repository.

1. Git Commands:

- **git version** : The command git version is used to check the version of git.

git --version

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git --version
git version 2.55.0.windows.3

```

- **git config**: Configures Git settings. Commonly used to set up user information

git config --global user.name "Your Name"

git config --global user.email "youremail@example.com"

- **git config --list**: Displays all the Git configurations for the current user.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git config --global user.name "Umang-SE-Lab"

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git config --global user.email "umang.se.lab@gmail.com"

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git config --global --list
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
user.name=Umang-SE-Lab
user.email=umang.se.lab@gmail.com
```

2. Repository Management

- **git init**: Initializes a new Git repository in the current directory

git init

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git init
Reinitialized existing Git repository in C:/SE Lab Internal Practice Week 2/.git/
```

- **git clone**: Creates a copy of an existing Git repository from a remote source (e.g., GitHub) to your local machine

git clone <https://github.com/username/repository.git>

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git clone https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
Cloning into 'SE-Lab-Week-2-Practice'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
```

- **git remote -v**: To see the remote repository that is connected to your local repository

git remote -v

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote -v
origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice.git (fetch)
origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice.git (push)
```

- **git remote add**: To add a new remote repository to your local repository

git remote add origin <https://github.com/username/repository.git>

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (master)
$ git remote add origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice.git
```

- **Pushing Changes to Remote:** To push your local commits to a remote repository:

`git push <remote_name> <branch_name>`

git push origin main

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 32 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 296 bytes | 296.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- **Pulling Updates from Remote:** To pull the latest changes from the remote repository and merge them into your local branch

`git pull <remote_name> <branch_name>`

git pull origin main

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 983 bytes | 245.00 KiB/s, done.
From https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
 ab957a8..f3c6b98  main      -> origin/main
Updating ab957a8..f3c6b98
Fast-forward
 file2.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git pull origin main
From https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
 * branch      main      -> FETCH_HEAD
Already up to date.
```

- **git remote remove:** To remove a remote repository from your local configuration:

`git remote remove <remote_name>`

git remote remove origin

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote remove origin
```

- **Renaming a Remote:** to rename an existing remote:

```
git remote rename <old_name> <new_name>
```

git remote rename origin upstream

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote rename origin upstream
```

- **Fetching Updates from Remote:** To fetch updates from the remote repository but not merge them into your local branch

git fetch <remote_name>

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch upstream
From https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
* [new branch]      main      -> upstream/main

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch upstream
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 951 bytes | 190.00 KiB/s, done.
From https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
f3c6b98..b53ee73  main      -> upstream/main
```

- **Changing Remote URL :** To change the URL of a remote (e.g., after changing the remote repository address):

```
git remote set-url <remote_name> <new_url>
```

git remote set-url origin <https://github.com/username/new-repository.git>

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote set-url origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
```

- **Viewing the Remote Repository's Information:** to see detailed information about a remote repository

```
git remote show <remote_name>
```

git remote show origin

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote show origin
* remote origin
Fetch URL: https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
Push URL: https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
HEAD branch: main
Remote branch:
  main tracked
Local ref configured for 'git push':
  main pushes to main (local out of date)
```

3. Staging and Committing

- **git status:** Shows the status of changes in your working directory and staging area. It tells you which files are untracked, modified, or ready to be committed.

git status

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  SE-Lab-Week-2-Practice/

nothing added to commit but untracked files present (use "git add" to track)
```

- **git add:** Adds changes in the working directory to the staging area.

git add filename.txt # Adds a specific file

git add . # Adds all changes in the directory

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git add .
warning: adding embedded git repository: SE-Lab-Week-2-Practice
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> SE-Lab-Week-2-Practice
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached SE-Lab-Week-2-Practice
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
warning: in the working copy of 'file3.txt', LF will be replaced by CRLF the next time Git touches it
```

- **git commit:** Commits the staged changes to the repository with a descriptive message. The -m option allows you to include a commit message.

git commit -m "Commit message describing changes"

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git commit -m "third file commit"
[main 30a8127] third file commit
2 files changed, 2 insertions(+)
create mode 160000 SE-Lab-Week-2-Practice
create mode 100644 file3.txt
```

4. Branching and Merging

- **git branch**: Lists all branches or creates a new branch

git branch # Lists all branches or

git branch -a # Lists all branches

git branch branch-name # Creates a new branch

git branch -d <branch_name> # Deletes a branch

Use -D to force-delete if it hasn't been merged.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch
* main

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -a
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch br1

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -d br1
Deleted branch br1 (was 30a8127).
```

- **Listing All Remote Branches** : To you want to list all branches on the remote repository

git branch -r

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -r
origin/HEAD -> origin/main
origin/main
```

- **git checkout**: Switches to a different branch

git checkout branch-name # Switches to an existing branch

git checkout -b new-branch # Creates and switches to a new branch

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch br1

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git checkout br1
Switched to branch 'br1'

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (br1)
$
```

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git checkout -b br1
Switched to a new branch 'br1'

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (br1)
$ |
```

- **Pruning Deleted Remotes** : If a branch was deleted on the remote but still shows up locally.

git remote prune origin

When someone **deletes a branch on the remote**, your local Git doesn't automatically remove the corresponding remote-tracking branch (like origin/old-feature).

git remote prune origin removes these outdated references.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote prune origin
```

- **Fetching a Specific Remote Branch**: To fetch a specific branch from a remote.
git fetch <remote_name> <branch_name>

git fetch origin feature-branch

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch origin br1
From https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
* branch          br1          -> FETCH_HEAD
* [new branch]     br1          -> origin/br1
```

- **Setting the Upstream Branch for Pushing** : When pushing for the first time and want to set the remote branch you are pushing to:

git push --set-upstream <remote_name> <branch_name>

git push --set-upstream origin feature-branch

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git push --set-upstream origin br1
error: src refspec br1 does not match any
error: failed to push some refs to 'https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice'
```

- **git merge**: Merges the specified branch into the current branch. This command integrates the changes from the feature branch into the main branch.

git checkout main # Switch to the main branch

git merge branch-name # Merge branch-name into main

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (br1)
$ git checkout main
branch 'main' set up to track 'origin/main'.
Switched to a new branch 'main'

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git merge br1
Already up to date.
```


- **Rebasing a Local Branch onto a Remote Branch:** If you want to rebase your local branch onto a remote branch (this can be useful to keep your history linear)

```
git fetch <remote_name>
git rebase <remote_name>/<branch_name>
```

Example:

```
git fetch origin
git rebase origin/main
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch origin
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git rebase origin/main
Current branch main is up to date.
```

5. Undoing Changes

- **git reset** : Removes the specified file from the staging area but leaves the working directory unchanged. `git reset --hard` can also reset the working directory and staging area to the last commit.

git reset <file>:

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ nano file1.txt
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git add file1.txt
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git reset file1.txt
Unstaged changes after reset:
M   file1.txt
```

- **git revert**: Creates a new commit that undoes the changes from a specified commit, leaving the history intact.

git revert <commit>:

```
[main 5bb463f] Revert "fourth commit"
2 files changed, 2 deletions(-)
delete mode 100644 file4.txt
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ |
```

6. Viewing History

- **git log**: Shows a history of commits in the repository, including commit hashes, messages, and timestamps. Use `git log --oneline` for a more concise view.

git log:

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git log
commit 5bb463f1c2120de131a54fb73f1458644e29fd4f (HEAD -> main)
Author: Umang-SE-Lab <umang.se.lab@gmail.com>
Date: Sun Sep 6 12:44:39 2026 +0530

    Revert "fourth commit"

    This reverts commit b2a9558537dea2c4e114edd280faab381c0353ac.

commit b2a9558537dea2c4e114edd280faab381c0353ac (origin/main, origin/HEAD)
Author: Umang-SE-Lab <umang.se.lab@gmail.com>
Date: Sun Sep 6 12:43:24 2026 +0530

    fourth commit

commit 30a8127291aa1d7d9c7416eb2042cf1b10048708 (origin/br1, br1)
Author: Umang-SE-Lab <umang.se.lab@gmail.com>
Date: Sun Sep 6 12:04:31 2026 +0530

    third file commit

commit f3c6b98f009f68d452e5de8cd3fcd4bc109290f0
Author: Umang-SE-Lab <umang.se.lab@gmail.com>
Date: Sun Sep 6 11:57:59 2026 +0530

    Add note about change from GitHub to file2.txt

commit ab957a8ddb8a72ef04250f93e7794f83818f9eb7
Author: Umang-SE-Lab <umang.se.lab@gmail.com>
Date: Sun Sep 6 11:51:55 2026 +0530

    First commit

```

- **git diff**: Displays differences between various commits, the working directory, and the staging area. `git diff` without arguments shows changes not yet staged.

git diff:

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git diff

```

- **git show**: Shows the details of a specific commit, including the changes made and the commit message.

git show <commit>

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git show
commit 5bb463f1c2120de131a54fb73f1458644e29fd4f (HEAD -> main)
Author: Umang-SE-Lab <umang.se.lab@gmail.com>
Date: Sun Sep 6 12:44:39 2026 +0530

    Revert "fourth commit"

    This reverts commit b2a9558537dea2c4e114edd280faab381c0353ac.

diff --git a/file1.txt b/file1.txt
index 7d4cc7b..09c4868 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,3 +1,2 @@
 This is file1 week 2
-Changes done in GitBash

diff --git a/file4.txt b/file4.txt
deleted file mode 100644
index 18e3348..0000000
--- a/file4.txt
+++ /dev/null
@@ -1 +0,0 @@
-This is a new file for committing

```

7. To undo the changes made to file before staging it.

- **git restore filename**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git restore file3.txt

```

8. To correct committed with the wrong message

- **git commit --amend -m "Corrected commit message"**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git commit --amend -m "Corrected commit message"
[main add4ae9] Corrected commit message
Date: Sun Sep 6 12:44:39 2026 +0530
2 files changed, 2 deletions(-)
delete mode 100644 file4.txt

```

9. Recovering deleted branches

- **git reflog**
- **git checkout -b feature-ui <commit_hash>**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git reflog
add4ae9 (HEAD -> main) HEAD@{0}: commit (amend): Corrected commit message
5bb463f HEAD@{1}: revert: Revert "fourth commit"
b2a9558 (origin/main, origin/HEAD) HEAD@{2}: commit: fourth commit
30a8127 (origin/br1, br1) HEAD@{3}: checkout: moving from br1 to main
30a8127 (origin/br1, br1) HEAD@{4}: Branch: renamed refs/heads/main to refs/heads/br1
30a8127 (origin/br1, br1) HEAD@{6}: Branch: renamed refs/heads/br1 to refs/heads/main
30a8127 (origin/br1, br1) HEAD@{8}: checkout: moving from main to br1
30a8127 (origin/br1, br1) HEAD@{9}: Branch: renamed refs/heads/br1 to refs/heads/main
30a8127 (origin/br1, br1) HEAD@{11}: checkout: moving from main to br1
30a8127 (origin/br1, br1) HEAD@{12}: commit: third file commit
f3c6b98 HEAD@{13}: pull: Fast-forward
ab957a8 HEAD@{14}: commit (initial): First commit

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git checkout -b br1 30a8127
fatal: a branch named 'br1' already exists

```

10. To download the latest changes from the remote without merging

- **git fetch origin**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch origin

```

11. To remove accidentally committed sensitive file from Git history.

- **git filter-branch --force --index-filter **
- **"git rm --cached --ignore-unmatch secrets.txt" **
- **--prune-empty --tag-name-filter cat -- --all**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git filter-branch --force --index-filter
WARNING: git-filter-branch has a glut of gotchas generating mangled history
rewrites. Hit Ctrl-C before proceeding to abort, then use an
alternative filtering tool such as 'git filter-repo'
(https://github.com/newren/git-filter-repo/) instead. See the
filter-branch manual page for more details; to squelch this warning,
set FILTER_BRANCH_SQUELCH_WARNING=1.
Proceeding with filter-branch...

usage: git filter-branch [--setup <command>] [--subdirectory-filter <directory>] [--env-filter <command>]
      [--tree-filter <command>] [--index-filter <command>]
      [--parent-filter <command>] [--msg-filter <command>]
      [--commit-filter <command>] [--tag-name-filter <command>]
      [--original <namespace>]
      [-d <directory>] [-f | --force] [--state-branch <branch>]
      [--] [<rev-list options>...]
gitgit gitgit gitgit
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git rm --cached --ignore-unmatch secrets.txt
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ --prune-empty --tag-name-filter cat -- --all
bash: --prune-empty: command not found
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ --prune-empty --tag-name-filter cat-- --all
bash: --prune-empty: command not found

```

12. To merge changes from another branch

First switch to branch where changes are to applied and then merge another-branch

- **git checkout previous-branch**
- **git merge another-branch**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git checkout main
Already on 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git merge br1
Already up to date.

```

13. To resolve a merge conflict manually

You tried to merge two branches and Git reported a conflict in x.js. Then for conflict resolution do:

1. **Open x.js and resolve/remove the conflict markers** (<<<<<<, =====, >>>>>>)
2. After resolving:
 - ✓ **git add app.js**
 - ✓ **git commit # If Git didn't auto-create a merge commit**

14. The .gitignore

The .gitignore file is used to tell Git which files or directories to ignore in your project. It's a useful way to avoid committing unnecessary files like log files, build outputs, and IDE configurations.

a. Create a .gitignore File

In the root directory of your Git project, create a file called .gitignore

`touch .gitignore`

b. Add Rules to .gitignore

Inside the .gitignore file, you add patterns for the files and directories you want Git to ignore. Each pattern should be written on a new line.

Here are some common examples:

- Ignore all .log files: ***.log**
- Ignore a specific file: **secret_file.txt**
- Ignore all files in a temp/ directory: **temp/**
- Ignore all files except important_file.txt inside a folder:

folder/*
!folder/important_file.txt

- Ignore files with a specific extension: ***.bak**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ touch f1.txt f2.txt f3.log

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ touch .gitignore

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        f1.txt
        f2.txt
        file4.txt

nothing added to commit but untracked files present (use "git add" to track)
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$

```

15. To see who changed a particular line in a file

- **git blame script.py**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git blame file1.txt
^ab957a8 (Umang-SE-Lab 2026-09-06 11:51:55 +0530 1) This is file1 Week 2
^ab957a8 (Umang-SE-Lab 2026-09-06 11:51:55 +0530 2)

```

16. Git stash

You modified files A and B, but not committed them.

git stash # saves changes and reverts to clean state

git switch another-branch

do something else...

git switch main

git stash apply # brings back your changes

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git stash
No local changes to save

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git switch br1
Switched to branch 'br1'
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (br1)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git stash apply
No stash entries found.

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ |

```

17. To check branch is merge

- **git branch --merged**

If branch not merged then displays its name

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch --merged
br1
* main

```

18. To Delete multiple local branches at once

- **git branch -d branc1 branch2 branch3**

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -d br1
Deleted branch br1 (was 30a8127).

```

Task 3: Students must explore all git commands on given scenario-based question. (Note student write command in observation book and paste the answer (git command) below each question for uploading).

Scenario based questions on basic git commands

1. You made changes to one file in the above project but haven't staged them yet. You realize they were a mistake. What Git command will you use to discard the local changes?

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git restore file1.txt

```

2. You accidentally ran `git add file1.txt` (note instead of `file.txt` consider one file from above project), but you're not ready to commit yet. How do you remove it from the staging area without losing the changes?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git restore --staged file1.txt

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git reset file1.txt
```

3. You made a commit but typed the wrong commit message. You haven't pushed it yet. How do you fix it?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git commit --amend -m "New commit message"
[main d0fdf23] New commit message
Date: Sun Sep 6 12:44:39 2026 +0530
2 files changed, 2 deletions(-)
delete mode 100644 file4.txt
```

4. You want to view the commit history of the current branch in a readable format. What Git command should you use?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git log --oneline --graph --decorate
* d0fdf23 (HEAD -> main) New commit message
* b2a9558 (origin/main, origin/HEAD) fourth commit
* 30a8127 (origin/br1) third file commit
* f3c6b98 Add note about change from GitHub to file2.txt
* ab957a8 First commit
```

5. After cloning a repo, how can you set your name and email globally for all Git repositories?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git config --global user.name "Umang-SE-Lab"

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git config --global user.email "umang.se.lab@gmail.com"
```

6. You've made some edits to your files but haven't staged them. How can you view the changes?


```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git diff
diff --git a/file1.txt b/file1.txt
index 09c4868..217176d 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,2 +1,2 @@
 This is file1 week 2
-
+New change

```

7. You're on the main branch but need to switch to feature/login. What command do you

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git switch feature/login
Switched to branch 'feature/login'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (feature/login)
$

```

8. You deleted the feature-ui branch by mistake. You haven't pushed the deletion. How

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git reflog
b2a9558 (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: checkout: moving from feature/login to main
84270eb (feature/login) HEAD@{1}: checkout: moving from main to feature/login
b2a9558 (HEAD -> main, origin/main, origin/HEAD) HEAD@{2}: checkout: moving from feature/login to main
84270eb (feature/login) HEAD@{3}: commit: commit
d0fdf23 HEAD@{4}: Branch: renamed refs/heads/main to refs/heads/feature/login
d0fdf23 HEAD@{6}: commit (amend): New commit message
add4ae9 HEAD@{7}: checkout: moving from br1 to main
30a8127 (origin/br1) HEAD@{8}: checkout: moving from main to br1
add4ae9 HEAD@{9}: checkout: moving from main to main
add4ae9 HEAD@{10}: commit (amend): Corrected commit message
5bb463f HEAD@{11}: revert: Revert "fourth commit"
b2a9558 (HEAD -> main, origin/main, origin/HEAD) HEAD@{12}: commit: fourth commit
30a8127 (origin/br1) HEAD@{13}: checkout: moving from br1 to main
30a8127 (origin/br1) HEAD@{14}: Branch: renamed refs/heads/main to refs/heads/br1
30a8127 (origin/br1) HEAD@{16}: Branch: renamed refs/heads/br1 to refs/heads/main
30a8127 (origin/br1) HEAD@{18}: checkout: moving from main to br1
30a8127 (origin/br1) HEAD@{19}: Branch: renamed refs/heads/br1 to refs/heads/main
30a8127 (origin/br1) HEAD@{21}: checkout: moving from main to br1
30a8127 (origin/br1) HEAD@{22}: commit: third file commit
f3c6b98 HEAD@{23}: pull: Fast-forward
ab957a8 HEAD@{24}: commit (initial): First commit

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git checkout -b br1 30a8127
Switched to a new branch 'br1'

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (br1)
$

```

9. You've made some commits locally and now want to upload them to the remote repository. What do you run?

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date

```

10. How can you fetch the latest changes from the remote repository without merging them automatically?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch
```

11. You're on the main branch and want to start working on a new feature called search-filter. What command do you use?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git checkout -b search-filter
Switched to a new branch 'search-filter'

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (search-filter)
$
```

12. You committed a file containing an API key and want to completely remove it from the repo's history. What should you do?

```
Rog-Strix MINGW64 ~/project (main)
git filter-repo --path secrets.txt --invert-paths
Enumerating objects: 100% (15/15), done.
Counting objects: 100% (3/3), done.
Compressing old objects
Cleaning up
is now at 9a8b7c1 Clean history

git push --force
https://github.com/user/repo.git
b4d5e...9a8b7c1 main -> main (forced update)

Rog-Strix MINGW64 ~/project (main)
```

Scenario 1: It was your very last commit

If you just committed the file and haven't made any other commits since, you can pull it out of the commit and rewrite it:

1. Remove the file from Git's tracking but keep it on your hard drive:

Bash

```
git rm --cached <file-name>
```

2. Add the file to your .gitignore so it doesn't happen again.
3. Amend your last commit to overwrite it:

Bash

```
git commit --amend
```

Scenario 2: It is buried deep in your commit history

If you have made several commits since you accidentally added the key, you have to rewrite your entire project history.

Git's official recommendation for this is to use a dedicated tool called **git filter-repo** (or the **BFG Repo-Cleaner**). For example, using filter-repo, the command looks like this:

Bash

```
git filter-repo --path <file-name> --invert-paths
```

13. You want to see all the branches that exist both locally and on the remote. What

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (search-filter)
$ git branch -a
  br1
  feature/login
  main
* search-filter
  remotes/origin/HEAD -> origin/main
  remotes/origin/br1
  remotes/origin/main
```

14. You're on main and want to merge the completed feature/signup branch into it. What command do you use?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (search-filter)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git merge search-filter
Already up to date.
```

15. You tried to merge two branches and Git reported a conflict in app.js. What are the general steps to resolve it?

```
Strix MINGW64 ~/project (main)
  app.js

git -m "Resolve merge conflict in app.js"
8c9d] Resolve merge conflict in app.js
changed, 2 insertions(+), 1 deletion(-)

ge --continue
ceeded

Strix MINGW64 ~/project (main)
```

Step-by-Step Conflict Resolution

1. **Locate the Conflicted File**
 - Run git status to confirm that app.js is listed under "Unmerged paths."
2. **Open the File and Edit the Code**

- Open app.js in your code editor.
- Look for Git's conflict markers:

JavaScript

```
<<<<<<< HEAD
```

```
// Your current branch's code
```

```
=====
```

```
// The incoming branch's code
```

```
>>>>>>> branch-name
```

- Decide which code to keep (or combine both), then manually delete the <<<<<<<, =====, and >>>>>>> marker lines.

3. Stage the Resolved File

- Mark the conflict as resolved by adding app.js to the staging area:

Bash

```
git add app.js
```

4. Complete the Merge Commit

- Finalize the merge process:

Bash

```
git commit
```

- Git will automatically generate a default merge commit message (e.g., Merge branch '...' into ...). Save and close the editor (press Esc, type :wq, and hit Enter if it opens in Vim).

16. You don't want Git to track changes to .log files or node_modules/. How do you achieve this?

```
ASUS@Rog-Strix MINGW64 ~/project (main)
$ cat .gitignore

*.log
node_modules/

ASUS@Rog-Strix MINGW64 ~/project (main)
$
```

Add these patterns to your project's .gitignore file.

- Create the file if it does not already exist:

Bash

```
touch .gitignore
```

- Open .gitignore and add the following lines:

Plaintext

```
*.log
```

```
node_modules/
```

- If node_modules/ or any .log files were already committed in the past, remove them from Git's index without deleting your local copies:

Bash

```
git rm -r --cached node_modules
```

```
git rm --cached *.log
```

- Commit the updated .gitignore:

Bash

```
git add .gitignore
```

```
git commit -m "Ignore log files and node_modules"
```

17. You're investigating a bug and want to know who last changed line 25 in script.py. What command do you use?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git blame file1.txt
^ab957a8 (Umang-SE-Lab 2026-09-06 11:51:55 +0530 1) This is file1 week 2
b2a95585 (Umang-SE-Lab 2026-09-06 12:43:24 +0530 2) Chnages done in GitBash
^ab957a8 (Umang-SE-Lab 2026-09-06 11:51:55 +0530 3)

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git blame -L 2,2 file1.txt
b2a95585 (Umang-SE-Lab 2026-09-06 12:43:24 +0530 2) Chnages done in GitBash
```

Pro-Tip: If you want to tell Git to show you *only* line 25 so you don't have to scroll through a massive file, you can use the -L (line)

18. You're in the middle of working on a file but need to switch branches quickly. What do you do to save your work?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git stash
No local changes to save
```

When you are ready to resume working on those files later, you simply switch back to your original branch and run git stash apply (or git stash pop) to bring your unfinished changes right back into your text editor.

19. You previously ran git stash and now want to restore those changes. What command do you use?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git stash apply
No stash entries found.
```

Or git stash pop

20. The feature/test branch is no longer needed. How do you delete it locally?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -d search-filter
Deleted branch search-filter (was b2a9558).
```

21. You just merged the feature-ui branch into main. You want to clean up your local branches. How do you safely delete feature-ui?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -d search-filter
Deleted branch search-filter (was b2a9558).
```

22. You created a branch feature-experiment, made some changes, but now realize the code is no longer needed. You want to delete it, even though it hasn't been merged. What command will you use?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -D br1
Deleted branch br1 (was 30a8127).
```

23. You're currently on the feature-login branch and want to delete feature-ui. What must you ensure before running the delete command?

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (feature-login)
$ git branch -d feature-ui
Deleted branch feature-ui (was b2a9558).
```

To safely delete the feature-ui branch using the standard command (`git branch -d feature-ui`), you must ensure **that its changes have already been merged** (usually into main or the branch you are currently on).

Since you are already on a different branch (feature-login), you have passed the first rule of Git deletion: *you cannot delete the branch you are currently standing on*.

However, if feature-ui has unmerged commits, Git's safety check will block the lowercase -d command and throw an error to protect your work. *(As we saw in the previous question, if you are absolutely sure you want to destroy that unmerged work, you would have to bypass the warning by forcing it with -D).*

24. You want to delete bugfix-footer, but you're unsure if it's been merged. What should you do before deleting it?

Before deleting it, you should verify if its changes have been successfully integrated into your current branch (usually main).

There are two ways to handle this:

Option 1: Check the merged list Run this command to see a list of all branches that have already been merged:

Bash

```
git branch --merged
```

If bugfix-footer shows up in the output, you know it is 100% safe to delete.

Option 2: Just rely on Git's built-in safety net (The easiest way) You can simply try to delete it using the lowercase -d command:

Bash

```
git branch -d bugfix-footer
```

If you are wrong and the branch *hasn't* been merged yet, Git will automatically block the command, throw an error, and protect your unmerged work from being deleted.

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (feature-login)
$ git checkout -b bugfix-footer
Switched to a new branch 'bugfix-footer'

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (bugfix-footer)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch --merged
  bugfix-footer
  feature-login
* main

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -d bugfix-footer
Deleted branch bugfix-footer (was b2a9558).

```

25. You finished working on feature-a, feature-b, and feature-c. All have been merged into main. How do you delete them in one command?

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -d feature-a feature-b feature-c
Deleted branch feature-a (was b2a9558).
Deleted branch feature-b (was b2a9558).
Deleted branch feature-c (was b2a9558).

```

Scenario based questions on working with remote repository using Git commands

1. Specify the git command when you're starting work on a project and need to clone a remote repository to your local machine.

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git clone https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
fatal: destination path 'SE-Lab-Week-2-Practice' already exists and is not an empty directory.

```

2. Specify the git command if you want to see the remotes that are connected to your local repository.

```

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote -v
origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice (fetch)
origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice (push)

```

3. Specify the git command if you want to add a new remote repository to your local repository.


```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote add upstream https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
error: remote upstream already exists.
```

4. Specify the git command to remove a remote repository from your local configuration:

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote remove upstream
```

5. Specify the git command if you want to rename an existing remote:

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote rename origin upstream
```

6. Specify the git command to fetch updates from the remote repository but not merge them into your local branch.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch origin

ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch
```

7. Specify the git command to pull the latest changes from the remote repository and merge them into your local branch.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git pull
Already up to date.
```

8. Specify the git command to push your local commits to a remote repository.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git push
Everything up-to-date
```

9. Specify the git command to when pushing for the first time and want to set the remote branch you are pushing to.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
```

10. Specify the git command to change the URL of a remote (e.g., after changing the remote repository address).

```
git remote set-url <remote-name> <new-url>
```

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote set-url origin https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
```

11. Specify the git command if you want to list all branches on the remote repository.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git branch -r
origin/HEAD -> origin/main
origin/br1
origin/main
```

12. Specify the git command if a branch was deleted on the remote but still shows up locally.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch --prune
```

13. Specify the git command to fetch a specific branch from a remote.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git fetch origin br1
From https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
* branch          br1          -> FETCH_HEAD
```

14. Specify the git command if you want to see detailed information about a remote repository.

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git remote show origin
* remote origin
Fetch URL: https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
Push URL: https://github.com/Umang-SE-lab/SE-Lab-Week-2-Practice
HEAD branch: main
Remote branches:
  br1 tracked
  main tracked
Local branches configured for 'git pull':
  feature/login merges with remote main
  main merges with remote main
Local ref configured for 'git push':
  main pushes to main (up to date)
```

15. Specify the git command if you want to rebase your local branch onto a remote branch (this can be useful to keep your history linear)

git rebase <remote-name>/<branch-name>

```
ASUS@Rog-Strix MINGW64 /c/SE Lab Internal Practice Week 2 (main)
$ git rebase origin/br1
Current branch main is up to date.
```

Conclusion:

Successfully uploading a multi-folder project to both public and private GitHub repositories demonstrates student's practical understanding of **core Git commands** and **remote repository management**. By handling diverse file types (e.g., images, Java files, web pages, and pom.xml) and pushing them to different remote repositories, students show proficiency in:

- Initializing and configuring Git repositories
- Staging, committing, and managing version history
- Connecting to and working with multiple remote repositories
- Handling Gitignore and tracking large files appropriately
- Applying best practices for public vs. private repository usage

Completing these tasks equips students with essential version control skills and the ability to collaborate securely and professionally. It also reflects their readiness to contribute effectively to real-world software projects where source code management, team coordination, and secure code sharing are vital.